

SISTEMAS OPERATIVOS

Unidad 8

Disco y Sistema de Archivos

Apunte de estudio · disco rígido y MBR · métodos de asignación · FAT · EXT2 · cálculos de parcial

Mapa de la unidad

Esta unidad va de adentro hacia afuera del almacenamiento secundario: primero cómo está organizado físicamente el disco (geometría y particiones MBR), después cómo el sistema operativo decide en qué bloques poner cada archivo (métodos de asignación), y por último dos sistemas de archivos concretos que implementan esas ideas: FAT (asignación enlazada) y EXT2 (asignación indexada con i-nodos).

Bloque	Pregunta que responde	Idea central
1. Disco rígido / MBR	¿Cómo se organiza físicamente el disco y cómo se divide?	Sector → pista → cilindro; MBR + tabla de particiones (4 entradas)
2. Archivos y directorios	¿Qué es un archivo y dónde se guardan sus atributos?	Archivo = datos + metadatos; el directorio guarda los atributos
3. Métodos de asignación	¿En qué bloques del disco pongo cada archivo?	Contigua, enlazada e indexada
4. Espacio libre	¿Cómo sé qué bloques están libres?	Mapa de bits, lista enlazada, agrupamiento, contador
5. FAT	¿Cómo implementa Windows/pendrives la asignación enlazada?	Cadena de clústeres enlazados en una tabla
6. EXT2	¿Cómo implementa Linux la asignación indexada?	I-nodos con apuntadores directos e indirectos
7. Cálculos de parcial	¿Cómo resuelvo los ejercicios numéricos?	Tamaño de clúster/FAT y tamaño máximo por i-nodo

1. Disco rígido y esquema MBR

1.1 Geometría y capacidad

El **sector** es la unidad mínima de información que un disco puede leer o escribir. Habitualmente mide **512 bytes**.

La capacidad depende de la geometría del disco (cabezas, cilindros y sectores):

Fórmulas de geometría

$N.^{\circ} \text{ de sectores} = N.^{\circ} \text{ de caras} \times N.^{\circ} \text{ de pistas por cara} \times N.^{\circ} \text{ de sectores por pista}$

$\text{Capacidad} = N.^{\circ} \text{ de cabezas} \times N.^{\circ} \text{ de cilindros} \times N.^{\circ} \text{ de sectores por pista} \times 512 \text{ bytes}$

1.2 Estructura lógica del disco MBR

MBR (Master Boot Record): es el **primer sector de todo el disco** (cabeza 0, cilindro 0, sector 1). Contiene la tabla de particiones y un pequeño programa de inicialización (bootstrap) que lee esa tabla y cede el control al sector de arranque de la partición activa. Si no hay partición activa, muestra un mensaje de error.

- **Espacio particionado:** espacio del disco ya asignado a alguna partición.
- **Espacio sin particionar:** espacio todavía no accesible porque no fue asignado a ninguna partición.

1.3 Particiones

El SO no trabaja con la unidad física directamente, sino con volúmenes (entidades lógicas). Cada volumen es una partición y se comporta como si fuera un disco independiente.

- Las particiones son divisiones de tamaño fijo que ocupan cilindros contiguos.
- Cada partición tiene implantado su propio sistema de archivos.
- Para usar un disco hay que crear al menos una partición.

Tipos: primarias, extendida y lógicas

Concepto	Reglas clave
Particiones primarias	Máximo 4 por disco MBR (incluida la extendida si existe). Solo las primarias se pueden activar; los SO suelen instalarse en una primaria.
Partición extendida	Como máximo 1. Ocupa una de las 4 entradas de la tabla. No tiene sentido activarla; sirve de contenedor.
Particiones lógicas	Se definen dentro de la extendida, sin límite de cantidad. Pueden llenarla por completo o dejar espacio libre.
Partición activa	Aquella a la que el Master Boot cede el control al arrancar. El SO de la partición activa es el que se carga.

MBR vs GPT

Concepto	Reglas clave
MBR (Master Boot Record): estándar más antiguo, hasta 4 particiones primarias, máx. 2 TB por partición.	
GPT (GUID Partition Table): estándar moderno usado con UEFI, sin esas limitaciones.	

1.4 Tabla de particiones y contenido del MBR

La tabla de particiones es una estructura de **64 bytes** con **4 entradas de 16 bytes** cada una. Es independiente del sistema operativo.

Desglose de los 512 bytes del sector MBR:

Tamaño	Contenido
446 bytes	Código ejecutable de arranque (bootstrap / código maestro de inicio)
16 bytes	1. ^a entrada de la tabla de particiones
16 bytes	2. ^a entrada de la tabla de particiones
16 bytes	3. ^a entrada de la tabla de particiones
16 bytes	4. ^a entrada de la tabla de particiones
2 bytes	Marca AA55 (hex). La BIOS la verifica para comprobar que existe un MBR válido

Tareas del código maestro de inicio: examina la tabla buscando la partición activa, localiza su sector de inicio, carga una copia en memoria y le transfiere el control.

Mensajes de error posibles: «Tabla de partición no válida», «Error al cargar el sistema operativo», «Falta del sistema operativo».

2. Archivos y directorios

2.1 Concepto de archivo

Un archivo está formado por su **contenido (datos)** y por **atributos que lo describen (metadatos)**.

Atributos típicos: nombre, tipo, ubicación en el dispositivo, tamaño, información de protección, fechas/horas e identificación del usuario.

2.2 Directorios

El **directorio** (o tabla de contenidos) es la estructura que organiza el acceso a los archivos y **almacena sus atributos**. Al igual que los archivos, es no volátil (se guarda en disco) y se carga a memoria cuando se necesita.

Implementación de directorios

Esquema	Cómo funciona	Pro / Contra
Lista lineal	Lista de nombres con punteros a los bloques de datos y demás atributos. Búsqueda secuencial.	Simple (un vector), pero lento con muchas entradas. Ordenarla mejora la búsqueda pero complica alta/baja.
Tabla hash	Del nombre del archivo se obtiene, por una función de dispersión, el puntero a su entrada en la lista lineal.	Búsqueda/creación/eliminación rápidas. Contras: colisiones, tamaño fijo, agrandar implica reubicar todo.

3. Métodos de asignación de espacio

Objetivo: asignar espacio a los ficheros de modo que el disco se aproveche eficazmente y se acceda rápido a los archivos. Hay tres métodos: **contigua, enlazada e indexada**.

3.1 Asignación contigua

Cada fichero ocupa un conjunto de bloques contiguos. La entrada de directorio guarda la dirección del bloque inicial y la longitud (n.º de bloques). Optimiza el movimiento de cabezas porque la lectura del siguiente bloque empieza donde terminó el anterior. Permite acceso secuencial y directo.

Dificultades: encontrar el hueco para crear el fichero (algoritmos *First Fit* y *Best Fit*: en aprovechamiento son equivalentes, pero el primero suele ser más rápido) y **determinar cuánto espacio reservar**.

- **Problema principal:** fragmentación externa. Solución: compactación (costosa).
- **Crecimiento lento:** si se preasigna de más, hay fragmentación interna.

Variante con extensiones (extends): se asigna un trozo contiguo y, cuando hace falta, se añade otra extensión (NTFS, XFS). Hoy la asignación contigua pura se usa sobre todo en sistemas de solo lectura, como ISO-9660 en CD-ROM.

3.2 Asignación enlazada

Cada fichero es una lista enlazada de bloques de disco. La entrada de directorio apunta al primer (y último) bloque; cada bloque contiene un puntero al siguiente. Esos punteros no son accesibles por el usuario: si el bloque es de 512 bytes y el puntero ocupa 4, el usuario ve bloques de 508 bytes útiles.

Ventajas: elimina la fragmentación externa y no exige declarar el tamaño por anticipado.

Desventajas: solo eficiente para acceso secuencial y gasto de espacio en punteros.

Por clústeres: agrupar bloques en clústeres reduce búsquedas y el tamaño de las estructuras, a costa de más fragmentación interna. Confiabilidad: si se pierde un puntero se corta la cadena; se mitiga con listas doblemente enlazadas o guardando nombre + n.º de bloque relativo en cada bloque (gasto extra).

3.3 Asignación indexada

Reúne todos los punteros en un único lugar: el bloque índice. Cada archivo tiene su bloque de índices, una matriz donde la entrada *i*-ésima apunta al *i*-ésimo bloque del archivo. El directorio guarda la dirección del bloque índice; al crear el archivo, todos los punteros se inicializan en nil.

- **Índice multinivel:** un índice de primer nivel apunta a bloques de índice de segundo nivel, que apuntan a los bloques de datos (se pueden usar varios niveles).
- **Esquema combinado (Unix):** el *i*-nodo guarda primero punteros directos (a bloques de datos) y luego punteros indirectos simples, dobles y triples (a bloques de enlaces).

3.4 Comparación de los tres métodos

Criterio	Contigua	Enlazada	Indexada
Estructura	Bloques contiguos	Lista de bloques enlazados	Bloque(s) índice con punteros
Acceso directo	Sí, eficiente	No (solo secuencial)	Sí, eficiente
Fragmentación externa	Sí (su gran problema)	No	No
Fragmentación interna	Posible (preasignación)	Posible (clústeres)	Posible (clústeres)
Declarar tamaño previo	Sí	No	No
Costo extra	Compactación	Espacio de punteros	Espacio del bloque índice

4. Administración del espacio libre

El SO necesita saber qué bloques están libres para asignarlos. Métodos habituales:

Método	Descripción
Mapa de bits	La memoria/disco se divide en unidades; cada bit indica libre u ocupada (1 ocupado, 0 libre, o al revés según convención).
Lista enlazada	Lista en la que cada nodo describe una partición o un hueco libre.
Por agrupamiento	El primer bloque libre actúa como «bloque índice» con las direcciones de n-1 bloques libres; su última entrada apunta al siguiente bloque índice, y así sucesivamente.
Por contador (recuento)	Cada entrada guarda el n.º del primer bloque libre y cuántos bloques contiguos libres le siguen.

5. Sistema de archivos FAT

FAT (File Allocation Table, Tabla de Asignación de Archivos) es la segunda generación de sistemas de archivos, posterior a la asignación contigua. Es la implementación clásica de la **asignación enlazada**.

5.1 Idea básica

- El disco se divide en clústeres.
- Existe una tabla FAT que indica qué clúster sigue a cuál.
- Un archivo es una cadena de clústeres enlazados.

Clúster: agrupamiento de sectores, siempre potencia de 2 (cada sector = 512 bytes).

Almacena el contenido de los archivos; su tamaño se fija en el formateo y depende del tamaño del disco y del tipo de FAT.

Versión	Uso típico
FAT12	Disquetes
FAT16	Discos pequeños (DOS)
FAT32	Pendrives, tarjetas SD

5.2 Componentes y funcionamiento

Componentes del filesystem: (1) Boot sector, (2) Tabla FAT —puede haber 2 copias—, (3) Directorio raíz, (4) Área de datos (clústeres).

Funcionamiento: cada entrada de directorio apunta al primer clúster del archivo. Dentro de la tabla, cada clúster es un eslabón que apunta al siguiente, formando la cadena. El último clúster contiene un valor especial (p. ej. FF FF FF 0F en FAT32) que marca el fin del archivo (EOF).

Entrada de directorio FAT (32 bytes)

Campo	Tamaño
Nombre de archivo (relleno con espacios)	8 bytes
Extensión (relleno con espacios)	3 bytes
Atributos	1 byte
Reservado	10 bytes
Hora de última modificación	2 bytes
Fecha de última modificación	2 bytes
Primer clúster del archivo	2 bytes
Tamaño del archivo	4 bytes

El byte de atributos marca, entre otras cosas: solo lectura, oculto, de sistema, etiqueta de volumen, subdirectorio y bit de archivo.

5.3 Ventajas y desventajas

Ventajas	Desventajas y problemas
Simplicidad: estructura muy fácil de implementar y entender.	Sobrecarga administrativa: hay que leer y actualizar varias estructuras separadas para localizar/mantener un archivo.
Alta compatibilidad: lo soportan casi todos los sistemas y dispositivos.	Fragmentación: sin fragmentación externa, pero el archivo puede quedar muy disperso, obligando a mover las cabezas constantemente.
Bajo consumo de recursos; ideal para sistemas embebidos (SD, pendrives).	Fragilidad y corrupción de metadatos: las estructuras están separadas y pueden desincronizarse; se la considera frágil.
Recuperación relativamente fácil (aunque lenta) con CHKDSK/Scandisk.	Recuperación lenta y posibles inconsistencias: archivos cruzados (cross-linked) y cadenas perdidas/huérfanas.

6. Sistema de archivos EXT2

EXT2 (segundo sistema de archivos extendido) es un sistema tipo Unix/POSIX usado en Linux. Implementa la **asignación indexada** mediante i-nodos.

6.1 Estructuras básicas

Estructura	Qué es / qué contiene
Superbloque	Ocupa un solo bloque. Guarda información global: tamaño de las partes del FS, total de i-nodos y de i-nodos libres, n.º de bloques y de bloques libres, etc.
I-nodo	Estructura índice por archivo/carpeta. Contiene id, tipo y tamaño, pero NO el nombre ni el del padre. Incluye los apuntadores directos e indirectos.
Bloque	Contiene id, datos y, en directorios, los punteros a los i-nodos de sus hijos; en archivos, el contenido.
Bitmaps	Sucesión de 1 y 0 que indica si cada bloque o i-nodo está ocupado (1) o libre (0). Hay bitmap de bloques y bitmap de i-nodos.

6.2 Estructura física: grupos de bloques

EXT2 divide la partición lógica en Grupos de Bloques (Block Groups). Cada grupo duplica información crítica para poder recuperar el FS ante fallos. Antes de los grupos está el sector de boot, con el código máquina para cargar el kernel al arrancar.

Cada grupo de bloques contiene, en orden:

Parte del grupo	Tamaño	Contenido
Superbloque	1 bloque	Copia del superbloque (igual para todos los grupos)
Descriptores	n bloques	Tabla de descriptores: n.º de bloque de los bitmaps, n.º de bloques/i-nodos libres y directorios asignados (igual para todos; EXT2 lee el del grupo 0)
Bitmap de bloques	1 bloque	Mapa de bits de los bloques del grupo (específico de cada grupo)
Bitmap de i-nodos	1 bloque	Mapa de bits de los i-nodos del grupo (específico)
Tabla de i-nodos	n bloques	I-nodos del grupo (específica)
Bloques de datos	n bloques	Datos de archivos y directorios (específico)

6.3 Apuntadores del i-nodo

Cada i-nodo guarda una lista de apuntadores: directos (apuntan directamente a bloques de datos) e indirectos (apuntan a estructuras/bloques de apuntadores). Los indirectos pueden ser simples, dobles y triples, lo que permite que un archivo crezca mucho sin reservar espacio por adelantado.

6.4 Directorios EXT2

Una entrada de directorio contiene:

Campo	Significado
inodo	Índice al vector de i-nodos (tabla de i-nodos del grupo) para esta entrada
long_registro	Tamaño en bytes de la entrada de directorio
long_nombre	Cantidad de caracteres del nombre del archivo
tipo de archivo	Tipo de la entrada (archivo, directorio, etc.)
nombre	Nombre de esta entrada de directorio

Entradas especiales: « . » apunta al i-nodo del directorio actual y « .. » al del directorio padre. El directorio raíz « / » se encuentra accediendo directamente al i-nodo número 2.

Algoritmo para crear el directorio raíz « / » (resumen)

1. Acceder al i-nodo n.º 2 y obtener los bloques con las entradas del directorio /.
2. Buscar en el bitmap de bloques el primer bloque libre.
3. Cargar metadatos en un i-nodo y en un bloque.
4. Enlazar el primer apuntador directo del i-nodo con el bloque.
5. Escribir en disco las estructuras (i-nodo y bloque) en las posiciones consultadas en el superbloque.
6. Actualizar los dos bitmaps (poner en 1 el bit del i-nodo y del bloque ocupados).
7. Actualizar el superbloque.

Para agregar un archivo el algoritmo es similar, pero el bloque de datos contiene el contenido del archivo; se calcula cuántos bloques hacen falta según el tamaño; primero se usan los apuntadores directos y, si no alcanzan, los indirectos. Cuando se ocupan todos los directos e indirectos, se llega al tamaño máximo del archivo.

7. FAT vs EXT2 — visión de conjunto

Aspecto	FAT	EXT2
Método de asignación	Enlazada (cadena de clústeres)	Indexada (i-nodos)
Estructura central	Tabla FAT con el siguiente clúster de cada uno	I-nodo por archivo con apuntadores directos/indirectos
Acceso directo	Posible leyendo la FAT, pero debe seguir la cadena	Directo y eficiente vía bloque índice
Organización del volumen	Boot + FAT (×2) + raíz + datos	Boot + grupos de bloques (superbloque, descriptores, bitmaps, tabla de i-nodos, datos)
Nombre del archivo	En la entrada de directorio	En la entrada de directorio, NO en el i-nodo
Uso típico	Pendrives, SD, embebidos, alta compatibilidad	Distribuciones Linux
Punto débil	Frágil; metadatos separados que se desincronizan	Sin journaling propio (no registra cambios antes de aplicarlos)

8. Métodos de cálculo para el parcial

Esta es la parte que más rinde en el examen. Conviene tener clarísimas dos familias de cuentas: las de FAT (tamaño de clúster y de la tabla) y las de i-nodos (tamaño máximo de archivo). Aquí va el procedimiento general y luego ejemplos resueltos de la guía.

8.1 Cálculos sobre FAT

Relaciones que hay que recordar

Entradas máximas de la FAT = $2^{(\text{bits de la FAT})}$. FAT16 $\rightarrow 2^{16} = 65\,536$ clústeres.

N.º de entradas reales = tamaño de la partición $\div$ tamaño del clúster.

Tamaño de clúster mínimo = tamaño de la partición $\div$ n.º máximo de entradas.

Tamaño de la FAT (en bytes) = (bits por entrada $\div 8$) $\times$ n.º de entradas.

1 clúster = (n.º de sectores por clúster) $\times$ 512 bytes.

Ejemplo A — Direccionamiento y tamaño de clúster (FAT16)

Enunciado: FAT16 con bloque de 1 KB: ¿cuánto se direcciona como máximo? Si la partición es de 2 GB, ¿tamaño mínimo de bloque?

Paso 1. Con FAT16 hay 2^{16} entradas. Con bloque de 1 KB: $2^{16} \times 1 \text{ KB} = 64 \text{ MB}$ direccionables.

Paso 2. Para 2 GB: tamaño de bloque $\geq 2 \text{ GB} \div 2^{16} = 2^{31} \div 2^{16} = 2^{15} = 32\,768 \text{ bytes} = 32 \text{ KB}$ por bloque.

Ejemplo B — Tamaño de la FAT en sectores (FAT32)

Enunciado: Disco de 20 GB, una partición FAT32, clúster = 16 sectores $\times$ 512 B. ¿Cuántos sectores ocupa cada copia de la FAT?

Paso 1 (n.º de entradas). $20 \text{ GB} \div (16 \times 512 \text{ B}) = 20 \times 2^{17} = 2\,621\,440$ entradas.

Paso 2 (tamaño de la FAT). Cada entrada FAT32 = 32 bits = 4 bytes $\Rightarrow 20 \times 2^{17} \times 4 \text{ B} = 20 \times 2^{19} \text{ bytes}$.

Paso 3 (en clústeres). $(20 \times 2^{19}) \div (16 \times 512) = 20 \times 2^6 = 1\,280$ clústeres.

Paso 4 (en sectores). $1\,280 \times 16 = 20\,480$ sectores.

Ejemplo C — Sectores por clúster a partir de un archivo

Enunciado: Archivo de 160 KB que ocupa 10 entradas de la FAT; sector = 512 B. ¿Sectores mínimos por clúster?

Paso 1. $160 \text{ KB} \div 512 \text{ B} = 320$ sectores ocupa el archivo.

Paso 2. $320 \text{ sectores} \div 10 \text{ entradas} = 32$ sectores por clúster como mínimo.

8.2 Cálculos sobre i-nodos (tamaño máximo de archivo)

Procedimiento general

1) Índices por bloque = tamaño de bloque $\div$ tamaño del puntero (en bytes).

2) Bloques de DATOS direccionables:

- cada directo = 1 bloque
- cada indirecto simple = (índices por bloque) bloques
- cada indirecto doble = (índices por bloque)² bloques
- cada indirecto triple = (índices por bloque)³ bloques

3) Tamaño máximo = (total de bloques de datos) × (tamaño de bloque).

4) Bloques de ENLACES (índice): los directos no consumen; cada indirecto simple consume 1; cada doble consume 1 + (índices por bloque); cada triple consume 1 + ipb + ipb².

5) Tamaño en disco = (bloques de datos + bloques de enlaces) × tamaño de bloque.

Ejemplo D — i-nodo con bloque de 1 KB y punteros de 32 bits

Enunciado: i-nodo con 2 directos, 2 indirectos simples y 1 indirecto doble; bloque = 1 KB; puntero = 32 bits.

Índices por bloque: $1 \text{ KB} \div 32 \text{ bits} = 1 \text{ KB} \div 4 \text{ B} = 2^{10} \div 2^2 = 2^8 = 256$ índices.

a) Bloques de enlaces (máximo): directos = 0; cada indirecto simple = 1 ($\times 2 = 2$); el indirecto doble = 1 + 256 = 257. Total = 2 + 257 = 259 bloques de índice.

b) Bloques de datos (máximo): 2 (directos) + 2 $\times$ 256 (simples) + 256² (doble) = 2 + 512 + 65 536 = 66 050 bloques.

c) Tamaño máximo: 66 050 $\times$ 1 KB = 66 050 KB.

d) Tamaño en disco: (66 050 + 259) $\times$ 1 KB = 66 309 KB.

Ejemplo E — Bloques de enlaces para un archivo dado

Enunciado: i-nodo con 5 directos, 3 indirectos simples, 2 dobles y 1 triple; bloque = 2 KB; puntero = 64 bits. Archivo de 1548 KB: ¿cuántos bloques de disco almacenan enlaces?

Índices por bloque: $2 \text{ KB} \div 64 \text{ bits} = 2 \text{ KB} \div 8 \text{ B} = 2^{11} \div 2^3 = 2^8 = 256$.

Bloques de datos del archivo: $1548 \text{ KB} \div 2 \text{ KB} = 774$ bloques.

Reparto: 5 directos cubren 5 bloques (0 enlaces). Los 3 indirectos simples cubren 3 $\times$ 256 = 768 bloques y consumen 3 bloques de enlaces. Van 5 + 768 = 773; falta 1 bloque, que se indexa con el primer indirecto doble: usa 2 bloques de enlaces (1 del nivel doble + 1 del simple, el mínimo para indexar un bloque).

Resultado: 3 + 2 = 5 bloques de enlaces.

Causas que limitan el tamaño de un archivo

Por el campo «tamaño» del descriptor: si tiene 4 bytes (32 bits), el máximo es 2³² bytes.

Por la cantidad de bloques alcanzables desde el descriptor/i-nodo (directos + indirectos).

El tamaño máximo real es el MENOR de ambos límites.

9. Checklist de repaso rápido

- ☐ Sé calcular capacidad del disco y descomponer los 512 bytes del MBR ($446 + 4 \times 16 + 2 =$ marca AA55).
- ☐ Distingo primaria / extendida / lógica y qué significa partición activa (máx. 4 primarias en MBR).
- ☐ Explico archivo = datos + atributos, y que el directorio guarda los atributos (lista lineal vs hash).
- ☐ Comparo asignación contigua, enlazada e indexada (fragmentación, acceso directo, declarar tamaño).
- ☐ Conozco los 4 métodos de espacio libre (mapa de bits, lista, agrupamiento, contador).
- ☐ FAT: cadena de clústeres, componentes, EOF, entrada de directorio de 32 bytes, ventajas/desventajas.
- ☐ EXT2: grupos de bloques, superbloque, descriptores, bitmaps, tabla de i-nodos; i-nodo NO guarda el nombre.
- ☐ Resuelvo cálculos de FAT (tamaño de clúster y de la tabla) y de i-nodos (índices por bloque, datos, enlaces, tamaño máximo).

Para practicar números: Guía Práctica N.º 7, ejercicios 5–14 (FAT e i-nodos) con sus respuestas oficiales. Si querés, te armo una hoja aparte solo con esos ejercicios resueltos paso a paso.